Button Detection

1. Button Detection Overview

Button is the most common human-machine interaction input device. The MCU judges button state (pressed/released) by reading GPIO input levels. This experiment uses the BOOT button (GPIO0) to control LED (GPIO43) on/off toggle, demonstrating GPIO input configuration, **software debouncing**, and state machine programming.

Each button press → executes one LED state toggle (ON↔OFF), holding down does not trigger repeatedly.

Typical Application Scenarios:

Application Type	Example
Digital Input	User buttons, switch state reading, sensor digital signals, rotary encoder
Pull-up Input	Button connected to GND, limit switch, reed switch, photoelectric switch
Debounce Processing	Mechanical buttons, relay contact detection, DIP switch

2. Core Concepts

2.1 GPIO Input Mode

When configured as input mode, pins present **high-impedance state**, with level determined by external circuit.

Key Role of Pull-up/Pull-down Resistors:

Configuration	GPIO Default Level	Button Pressed Level	Use Case
Internal Pull-Up	High (1)	Low (0)	One end of button connected to GND, conducts when pressed
Internal Pull-Down	Low (0)	High (1)	One end of button connected to VDD, conducts when pressed
No Pull-up/down	Uncertain (floating)	Uncertain	**Not recommended**, floating pins susceptible to interference

This experiment uses **internal pull-up** (`GPIO_PULLUP_ENABLE`), with one end of button connected to GND and the other end to GPIO0:

```
VDD ──→ Internal pull-up resistor (~45kΩ) ──→ GPIO0 ──→ Button ──→ GND

Button released: GPIO0 pulled to VDD → reads 1
Button pressed: GPIO0 directly connected to GND → reads 0
```

2.2 Mechanical Button Bounce

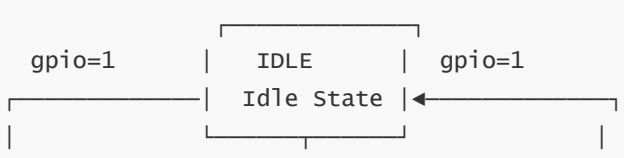
Mechanical buttons experience **bouncing** at the moment of press and release, generating a string of irregular high/low level pulses:

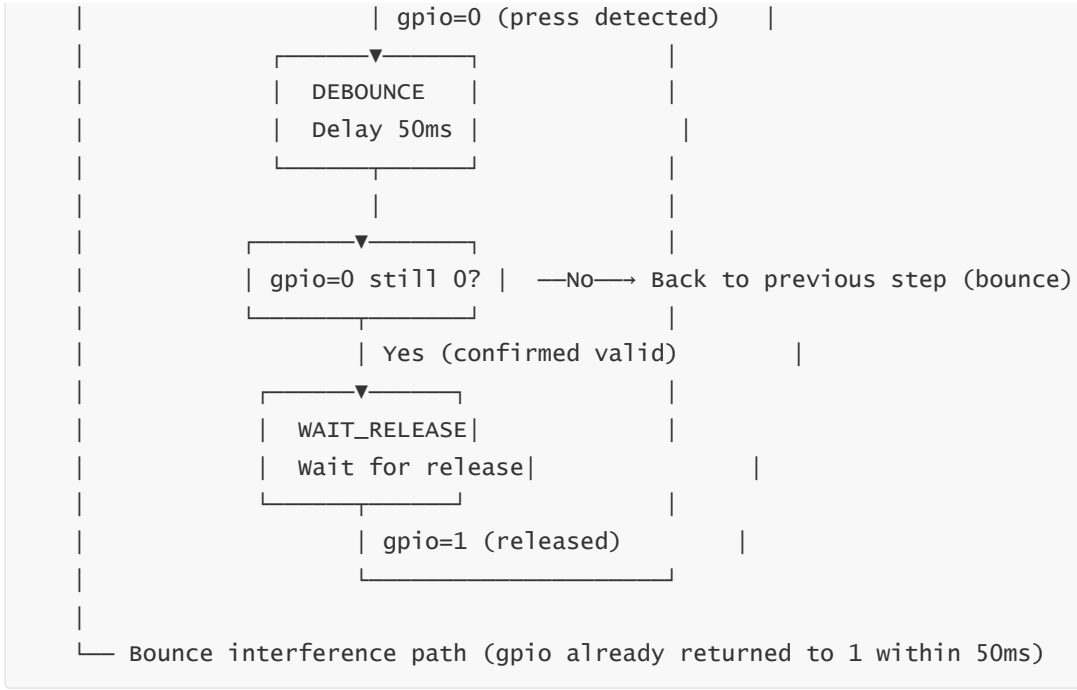
```
Ideal waveform:  ─────┐             ┌─────
                      └─────────────┘

Actual waveform:  ─────┐┌┐┌┐┌┐┌┐┌┐┌─────
                       └┘└┘└┘└┘└┘└┘
                       ├─ Bounce period ─┤
                      (typical 5~20ms)
```

If not debounced, MCU might misread one press as multiple triggers. **Software debounce** (delay and then confirm) is the simplest and most effective method.

2.3 Debounce State Machine





Meaning of four states:

State	Entry Condition	Operation	Output
IDLE	Previous processing complete	Detect gpio=0?	—
DEBOUNCE	First low level detected	`vTaskDelay(50ms)`	—
CONFIRM	Re-detect after 50ms	If still low=valid, otherwise=bounce	true/false
WAIT_RELEASE	Confirmed valid press	while loop waiting for gpio=1	Returns true only after release

---

3. Project Architecture and Flow

```
app_main()
  ├─ led_init()         ← Configure GPIO43 push-pull output
  ├─ key_init()         ← Configure GPIO0 input + internal pull-up
  ├─ led_control()      ← Initial state: LED ON
  |
  └─ while(1) Main Loop
      ├─ key_scan()     ← Debounced scan, returns bool
      |   ├─ false → skip this iteration
      |   └─ true  → led_state = !led_state (toggle)
      |              led_control()            (update GPIO)
      └─ vTaskDelay(20ms)
```

Key Design Principles:

- `led_state` saves **logical state** (true=ON, false=OFF)

- `led_control()` is the only function that operates GPIO, translating logical state to physical level
- `key_scan()` returns true means "one valid press completed (including release)"

---

4. Hardware Dependencies

Hardware Connection:

Pin	Peripheral	Configuration	Description
GPIO43	LED	Push-pull output	Common-anode, low level turns on
GPIO0	BOOT Button	Input + internal pull-up	Pressed=low, released=high

Dependencies: `freertos`, `driver/gpio`

---

5. Source Code Analysis

5.1 Macro Definitions and Global Variables

First define pin numbers and debounce time, then use a global variable to remember whether LED is currently ON or OFF.

```
#define LED_GPIO         43
// Button: GPIO0 with pull-up
#define KEY_GPIO          0
#define KEY_DEBOUNCE_MS  50              // Debounce time (ms)
static bool led_state = true;
```

> **Why is debounce time 50ms?** Mechanical button bouncing typically lasts 5~20ms. 50ms provides sufficient safety margin while being imperceptible to humans (humans are insensitive to button delays below 50ms). If debounce is too short (<10ms), it may not fully filter bouncing; if too long (>200ms), the button feels sluggish.

5.2 led_init() — LED Output Configuration

Configure GPIO43 as output mode to drive LED.

```
void led_init(void)
{
    gpio_config_t led_config = {
        .pin_bit_mask = 1ULL << LED_GPIO,
        // Push-pull output
        .mode         = GPIO_MODE_OUTPUT,
        .pull_up_en   = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type    = GPIO_INTR_DISABLE,
    };
    gpio_config(&led_config);
}
```

> This experiment configures GPIO directly in main (suitable for quick prototyping).

5.3 key_init() — Button Input Configuration

Configure GPIO0 as input mode with internal pull-up to detect button presses.

```
void key_init(void)
{
    gpio_config_t key_config = {
        .pin_bit_mask = 1ULL << KEY_GPIO,
        .mode         = GPIO_MODE_INPUT,            // Input mode
        // Internal pull-up
        .pull_up_en   = GPIO_PULLUP_ENABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type    = GPIO_INTR_DISABLE,
    };
    gpio_config(&key_config);
}
```

> **Why use internal pull-up instead of external pull-up resistor?** ESP32-S3 has built-in ~45kΩ weak pull-up, which is fully sufficient for button applications, saving BOM cost and PCB space. External pull-up (such as 10kΩ) is suitable for longer traces requiring more stable levels.

5.4 key_scan() — Debounced Scanning (Core)

Core logic of debounced scanning: detect low level → wait 50ms debounce → confirm still low → wait indefinitely for release → return true.

```c
bool key_scan(void)
{
    if (gpio_get_level(KEY_GPIO) == 0) {           // ① Detect LOW
        // ② Debounce delay 50ms
        vTaskDelay(KEY_DEBOUNCE_MS / portTICK_PERIOD_MS);
        if (gpio_get_level(KEY_GPIO) == 0) {       // ③ Re-confirm
            while (gpio_get_level(KEY_GPIO) == 0)   // ④ Wait for release
                vTaskDelay(10 / portTICK_PERIOD_MS);
            return true;                           // Valid press
        }
    }
    return false;                                  // No press or bounce
}
```

Execution flow analysis (for one complete button press):

```
Timeline         GPIO0 Level       key_scan() Behavior
──────────────────────────────────────────────────────
0ms            1 (released)      ①Read 1 → return false
...continuous polling...
100ms          0 (just pressed)   ①Read 0 → enter debounce
100~150ms      0 (may be bouncing) ②vTaskDelay(50ms) suspends
150ms          0 (stable)         ③Read again = 0 → confirmed valid
150~300ms      0 (holding)        ④while(0) loop waiting
300ms          1 (released)       ④while exits → return true
```

> **Why wait for release inside the debounce function?** If returning true directly without waiting, the main loop calls `key_scan()` again after 20ms and would detect low level → toggle again → one press triggers multiple times. Embedded wait-for-release ensures "one press = one toggle", while preventing repeated triggers during long press.

5.5 led_control() — State Output

According to global variable led_state, actually set GPIO43 high/low level. Whether LED is ON or OFF is ultimately decided here.

```c
void led_control(void)
{
    gpio_set_level(LED_GPIO, led_state ? 0 : 1);
}
```

> Centralizes mapping between logical state and physical level in this function. Currently common-anode connection (`? 0 : 1`, low level turns on). If hardware is common-cathode, just change `? 0 : 1` to `? 1 : 0`, no changes needed in any callers.

5.6 app_main() — Main Entry

Main function: first initialize LED and button to their positions, then in an infinite loop continuously scan button; once a valid press is detected, toggle LED state.

```
void app_main(void)
{
    led_init();
    key_init();
    led_control();

    while (1) {
        if (key_scan()) {                   // Valid press detected
            // Toggle logical state
            led_state = !led_state;
            // Update hardware output
            led_control();
        }
        vTaskDelay(20 / portTICK_PERIOD_MS); // 20ms polling interval
    }
}
```

> **Why 20ms polling interval:** FreeRTOS default tick is 100Hz (10ms). 20ms is approximately 2 ticks, ensuring reasonable button response delay (worst case: just missed poll at press moment → maximum 20ms delay). Also very low CPU usage.

---

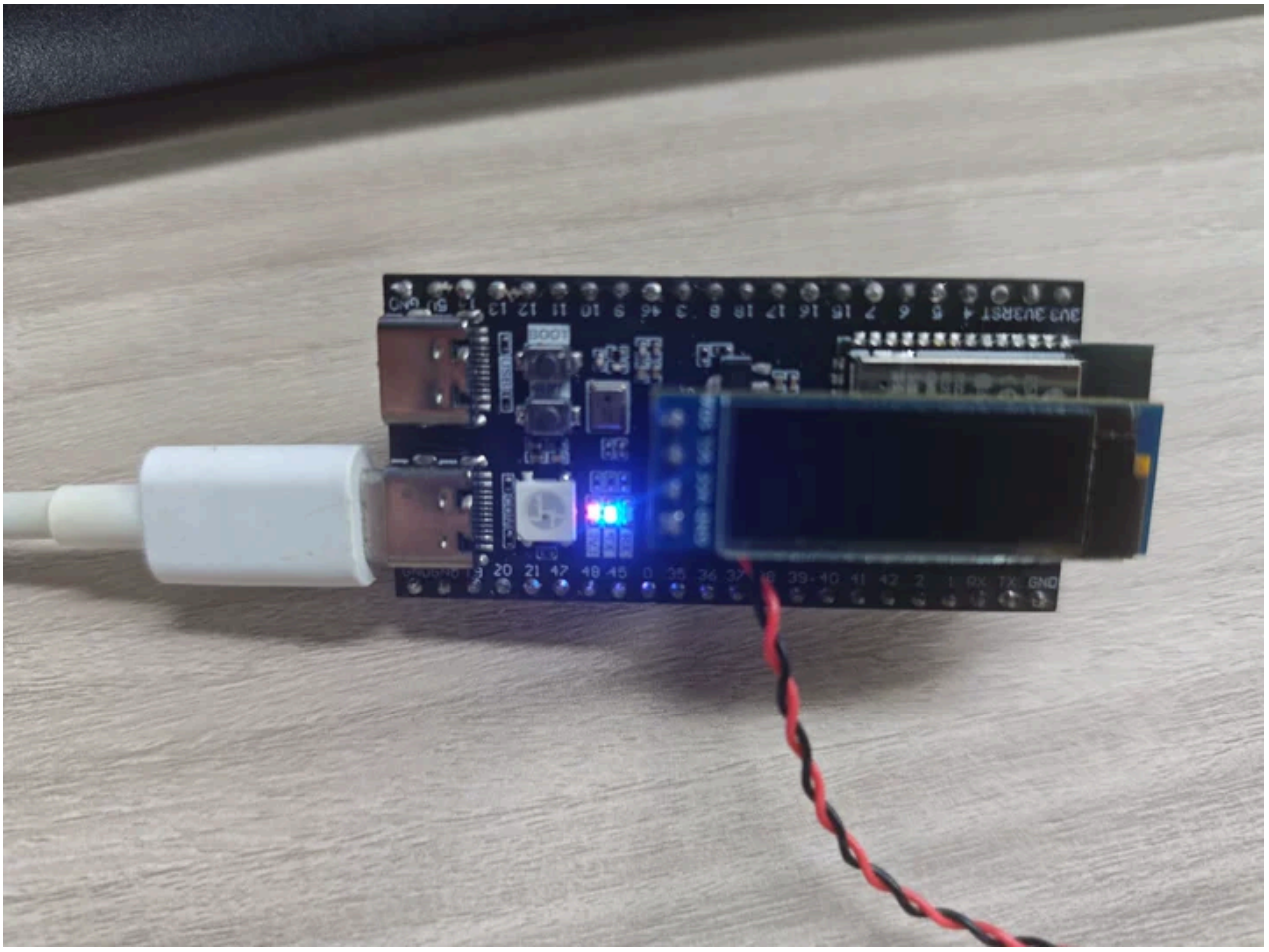
6. Operation Flow

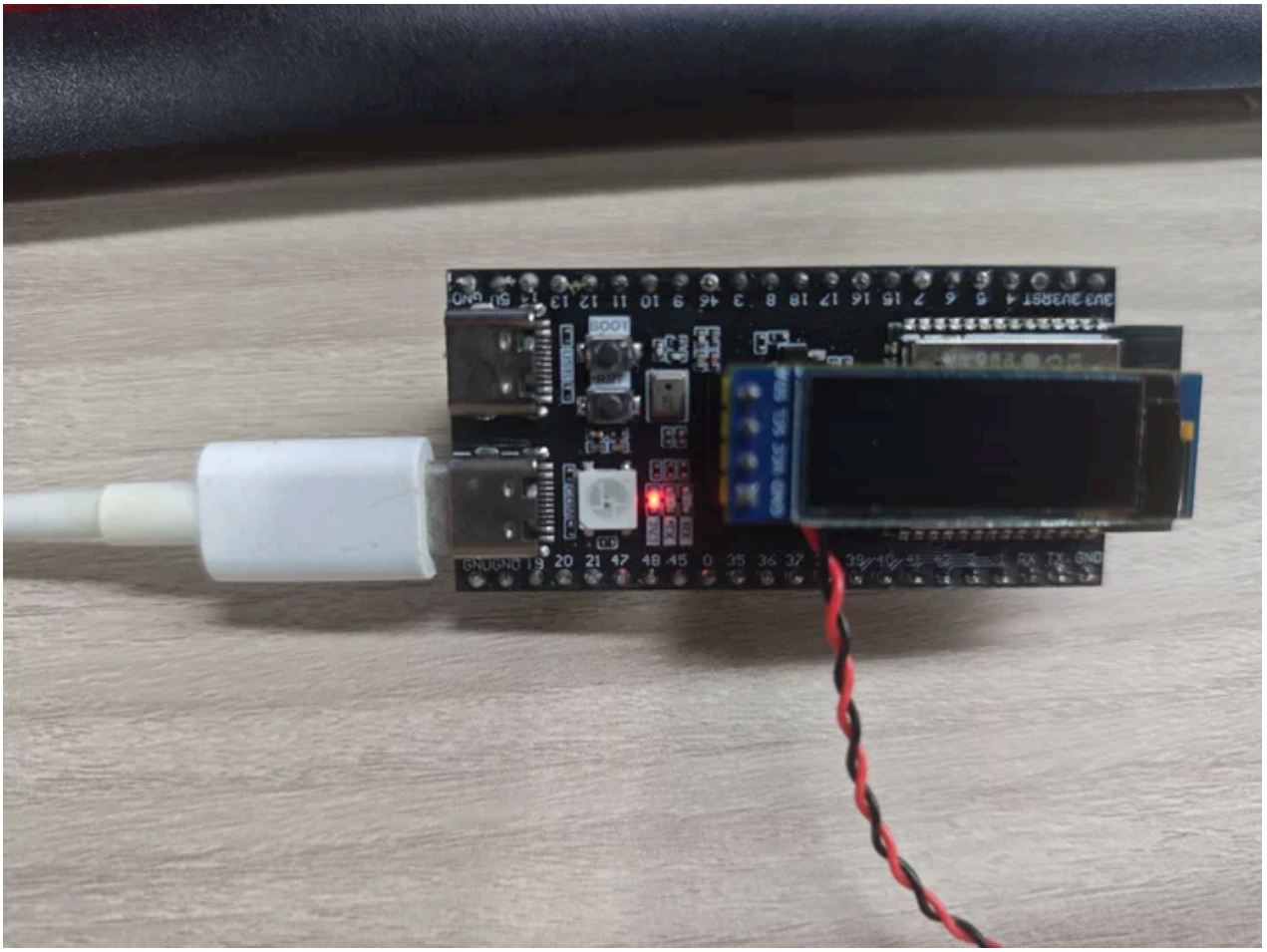
6.1 Build and Flash

> **For detailed flashing process, see: 1. Before Development → 3. Build and Flash**

6.2 Normal Operation

- LED initially **ON** after power-on
- Each press of BOOT button → LED toggles once (ON→OFF or OFF→ON)
- **Holding down**: Only toggles once, next toggle only after release





6.3 Boundary Testing

Test Item	Operation	Expected Result
Short press	Quickly click BOOT	LED toggles once
Long press	Hold BOOT button	LED toggles once (no repeated toggle while holding)
Rapid continuous press	Continuous fast clicks	Each valid press toggles once (may lose very short pulses due to debounce)
Power-on default	Power on without pressing button	LED ON (led_state initially true)

---

7. Common Issues

Phenomenon	Cause	Solution
One press triggers multiple toggles	Debounce time insufficient	Increase `KEY_DEBOUNCE_MS` (recommended 30~50ms)
Button response sluggish	Debounce time too long / main loop interval too large	Decrease `KEY_DEBOUNCE_MS` or main loop delay
Button completely unresponsive	GPIO0 misconfigured / no internal pull-up	Check `pull_up_en = GPIO_PULLUP_ENABLE`
Toggles by itself without pressing	GPIO0 floating + no pull-up/down → random level	Enable internal pull-up (`GPIO_PULLUP_ENABLE`)
LED and control logic reversed	Common-anode/common-cathode connection error	Modify ternary mapping in `led_control()`
Compilation error `driver/gpio.h` not found	Missing dependency component	Add `driver` to REQUIRES in CMakeLists.txt